Profile-Based Privacy-Preserving Extension for Internet Browsers

Rîsteiu Ioana-Stefania
Technical University of Cluj-Napoca, Faculty of Automation and Computer Science
Cluj-Napoca, Romania
ioana.stefania.risteiu@gmail.com

Prof. Coordinator Adrian Colesa
Department of Computer Science, Technical University of Cluj-Napoca, Romania
adrian.colesa@cs.utcluj.ro

Prof. Coordinator István-Attila Császár
Department of Computer Science, Technical University of Cluj-Napoca, Romania
istvan.csaszar@cs.utcluj.ro

***Abstract*—Browser fingerprinting enables websites to identify and track users across sessions without cookies by combining device and software properties into a unique identifier. In this paper, we present an anti-fingerprinting browser extension for Chromium browsers which replaces user's unique browser fingerprint with one that can be considered part of a group of artificially generated profiles. Rather than blocking fingerprinting APIs, which can be easily detected, the extension spoofs browser properties such as Navigator, Screen, Canvas, WebGL, Audio, Timezone, WebRTC, and Web Worker execution contexts. A seeded pseudo-random number generator ensures internal consistency within a session. The extension was evaluated by comparing fingerprint values across different users, demonstrating how each property changes between sessions.**

I. INTRODUCTION

Online tracking is constantly evolving beyond the era of third-party cookies. As browsers restrict or block persistent identifiers, advertisers and analytics providers have shifted their focus to tracking techniques that do not require storage on the user's device. Browser fingerprinting is the most widespread technique. It works by making multiple API calls to gather browser properties, such as screen dimensions, installed fonts, GPU renderer strings, and audio processing characteristics. After reading all the properties, a server can construct a near-unique identifier from how a particular browser and device combination renders content [1].

Unlike cookies, fingerprints cannot be deleted. They persist across private browsing sessions, often survive browser updates, and do not require user action to gather. Research indicates that even a small set of properties can uniquely identify many browser users [1,2].

Existing defences follow one of two strategies. The first, blocking, involves entirely preventing access to the fingerprinting API, such as returning null, throwing exceptions, or providing a fixed constant. This approach can be identified because the absence of a value, or a synthetic one, acts as an indicator that a privacy tool is active [3]. The second strategy is noise injection. This approach returns slightly different values each time a request is made, making consistent fingerprinting impossible. This method interferes with internal consistency, a site that requests the same property twice and receives different values each time can be easily flagged as trying to use anti-fingerprinting techniques.

This paper presents an alternative to these methods, profile pooling. Instead of blocking or randomising property values per request, the extension assigns each browser session a coherent fake identity selected from a pool of realistic profiles. Within a session, every API returns the same values, so the user identifier remains consistent throughout the active session. Across sessions, the profile changes, so two visits to the same website cannot be linked. More importantly, because profiles are drawn from real-world value distributions, individual users can easily blend into the already detected fingerprints rather than stand out as privacy-tool users.

The implementation takes the form of a Chrome Manifest V3 extension organised into two layers that work together. The first handles HTTP request headers, rewriting User-Agent, Sec-CH-UA, and Accept-Language through the *declarativeNetRequest* API. The second injects a content script at document start, intercepting JavaScript APIs before any page code executes. To cover Web Worker contexts, the extension uses a Blob URL mechanism to inject the same profile into Workers, so fingerprinting scripts running there cannot observe any inconsistencies with the main page.

Section 6 covers the evaluation methods. Among the approaches used, we implemented a custom testing framework that simulates five separate users, each starting a fresh browser session, and checks whether the spoofed values change across sessions as expected.

II. BACKGROUND AND RELATED WORK

A. Browser Fingerprinting Mechanisms

Eckersley's 2010 research [1] showed that browser fingerprinting is highly feasible. By combining data such as the User-Agent string, screen resolution, system fonts, timezone and plugin list, it was possible to uniquely identify 83.6% of browsers in a dataset of over 470000 visitors. The findings indicate that each property adds entropy bits, and the combined entropy of multiple properties typically exceeds 33 bits, which is sufficient to distinguish a single user from the global internet population.

Englehardt and Narayanan's 2016 study [2] analysed fingerprinting practices on a large scale, discovering that out of the top 100000 visited websites, 14371 are running canvas fingerprinting scripts. This technique involves rendering text

and shapes off-screen and hashing the pixel data. Variations in GPU rendering pipelines, anti-aliasing, and subpixel rendering lead to differences between devices, making the resulting hash a hardware-dependent identifier that remains consistent over time and is resistant to most user interventions.

A later study by Gomez-Boix [5] collected over 2 million fingerprints from a general news website and found that only 33.6% were unique, much lower than earlier results. This difference is likely because previous studies collected data from privacy-focused websites, where visitors tend to have more unusual browser setups. The finding shows that fingerprint uniqueness depends on the type of audience being measured.

Modern fingerprinting scripts have advanced beyond these techniques. WebGL graphics hardware descriptors expose the GPU model and driver version. The Web Audio APIs reveal numerical properties of the Digital Signal Processing (DSP) pipeline. Timezone APIs reveal geographic location. User Agent Data exposes the UA structure, including architecture, platform version and device model. Any of these may be combined to construct a fingerprint with high precision.

B. Existing Defences

Tor Browser [4] takes the most aggressive blocking approach, built around a single core principle: uniformity. Rather than spoofing or randomising values, Tor Browser offers a fixed, standardised configuration so that every user presents an identical fingerprint. If all users look the same, fingerprinting becomes useless, and no individual can be identified.

In practice, this means imposing a series of constraints. Some attributes, such as the operating system and language, are essential for basic functionality and cannot be fully hidden or spoofed. Rather than removing them, Tor Browser reduces variation in these attributes to make individual users less distinguishable. Font enumeration is limited, character fallback is applied, screen and window dimensions are standardised through letterboxing, and the set of accepted languages is restricted to a small, predefined list. However, it comes with significant costs. The fixed window size and disabled APIs cause usability problems on many websites, and the overall browsing experience is constrained.

Brave Browser [7] takes a softer version of the same approach, applying per-session randomization to canvas and audio outputs while keeping other properties close to real values. Unlike Tor Browser, Brave does not enforce a uniform identity across users, so the anonymity set is not well-defined.

Privacy Badger, uBlock Origin, and similar browser extensions block HTTP requests to domains known to host fingerprinting scripts, using blocklists such as EasyPrivacy or Disconnect.me. When a page attempts to load a third-party analytics or tracking script from a known domain, the extension intercepts the network request and prevents it from executing.

C. Our approach

Our approach is built on three properties. First, session consistency is achieved by using the same PRNG sequence, seeded at the start of the session, to select the spoofed value returned within a single browser session. Second, cross-session diversity is achieved because the seed is stored in “sessionStorage” and discarded when the tab closes, resulting in each new session having a different profile. A fingerprinting service that records a visitor's fingerprint cannot match it to a future visit. The third is profile credibility. The spoofed values are not arbitrary constants, they are drawn from pools of values observed in real browsers, covering common Chrome versions, GPU configurations, screen resolutions, and language settings. A user running the extension does not stand out as a privacy tool user.

III. DESIGN

A. Threat Model

We consider a tracking server that reads browser properties using standard JavaScript APIs, may call the same API multiple times within a session to test for inconsistency and may compare values across the HTTP header layer and the JavaScript layer to detect mismatches. Additionally, the server may inspect overridden functions using “toString()” function to detect extension modifications and run fingerprinting code inside Web Workers to get around content script injection.

B. Architecture

- **Layer 1** - HTTP Header Rewriting

  The background service worker manages HTTP-level spoofing. When the extension is activated in the browser, the service worker sets up a rewriting rule that intercepts all outgoing HTTP requests and replaces the headers with a chosen fake profile. These headers include information that the browser automatically sends with each request before the page loads: the browser name and version, structured identity fields encoding the same information in a machine-readable format, a flag indicating whether the device is mobile or desktop, the operating system name and the user's preferred language list. Together, these headers create a consistent identity that the server can read without executing any JavaScript. The extension rewrites all of these headers together because fingerprinting services check if there are mismatches between the header values. For example, the version numbers of each property must remain compatible with those of the others.

- **Layer 2** - JavaScript API Hooking

  The second component runs directly within the page and intercepts JavaScript API calls before any website code executes. It is injected at the earliest stage of page loading, before the browser parses or runs any scripts. This timing is essential, as fingerprinting scripts loading early could otherwise access real browser property values before protection is active. Once injected, the spoofing code installs modified browser APIs so that all succeeding scripts,

whether from the website or third-party trackers, only access the spoofed values.

C. Session Seed and Pseudo-Random number generator (PRNG)

Session consistency is achieved by storing a seed in a storage area associated with the current browser tab. During the first time the page runs in a tab, a short random string is generated using the browser's native randomness source, combining two generated numbers for unpredictability. The seed is reused across navigations within the same tab and lost when the tab closes, so the next session receives a different profile.

The seed is obtained using the Mulberry32 generator. Two 32-bit state values are first derived from it using polynomial rolling hashes with different multipliers:

$$a_0, b_0 = \sum_{i=1}^{n} c_i \cdot m^{n-i} \bmod 2^{32}, \quad m \in \{31,37\} \qquad (1)$$

On each call, both states advance by fixed odd increments, wrapping at 32 bits:

$$a_k = a_{k-1} + \delta_1, \quad b_k = b_{k-1} + \delta_2 \pmod{2^{32}} \qquad (2)$$

The states are then mixed and normalized into a uniform output in [0,1):

$$r_k = \frac{t \oplus (t \gg 14)}{2^{32}}, \text{ where } t = \left(a_k \oplus (a_k \gg 15)\right) \cdot (b_k | 1) \qquad (3)$$

Separate generator instances, seeded with distinct suffixes appended to the seed string, are used for canvas and audio noise, isolating them from the profile-selection sequence.

D. Value Pools

All spoofed values are drawn from pools of commonly seen profiles, selected based on available statistics provided by sites such as StatCounter [11], the Steam Hardware Survey [12], and the AmIUnique fingerprint dataset [13]:

- **UA profiles**: six Chrome versions on Windows 10 x64 (Chrome 137 through 142). Each entry groups the full UA string, navigator.platform, and the complete userAgentData structure including the shuffled brands array.
- **WebGL profiles**: seven GPU configurations drawn from the Steam Hardware Survey top entries: NVIDIA GeForce RTX 3060, RTX 4060 Laptop GPU, RTX 4060, RTX 3050, GTX 1650, Intel Iris Xe Graphics, and AMD Radeon RX 6600.
- **Memory**: [8, 16, 16, 16, 32, 32, 32] GB, weighted toward 16 GB (41%) and 32 GB (37%), reflecting the shift away from low-memory configurations in current hardware.
- **CPU cores**: [4, 6, 6, 6, 8, 8, 8, 10, 16], weighted toward 6 and 8 cores as the dominant configurations at approximately 28% and 27% respectively.
- **Languages**: seven European language lists (English, German, French, Italian, Spanish, Dutch, Polish), each maintaining the correct primary/secondary structure.
- **Timezone**: five Western/Central European zones with correct Internet Assigned Numbers Authority names, UTC offsets and GMT suffix strings.
- **Screen**: four common desktop resolutions (1920x1080, 1536x864, 1366x768, 2560x1440) with realistic taskbar deductions for available height.
- **Canvas fonts**: twelve common Windows system fonts used to vary canvas text rendering.

IV. Implementation

A. Function Masking

All API interception starts with overriding *Function.prototype.toString*. This step is important because fingerprinting scripts first check whether a function has been interfered or not. The check is simple: call *.toString()* on any browser function, and a real, untouched function will return a string containing *[native code]*. A modified function will return its actual JavaScript source code, immediately revealing that it has been replaced.

To prevent this, the extension maintains a Map called *hookedFunctions* that pairs each hooked function with the name it should report. Before any other hook is installed, the extension saves a reference to the original *Function.prototype.toString* and redefines it using *Object.defineProperty*. The new version checks the map every time it is called. If the target function is found in the map, it returns the expected string, which is exactly what the browser would return for a native function. If the function is not in the map, the call is passed through to the original *toString*, so that unrelated functions continue to behave normally.

Each hook is registered through a helper function called *hideFn(fn, name)*. This function does two things: it adds the function to the *hookedFunctions* map so that *toString* returns the right output, and it overwrites the function's name property using *Object.defineProperty* to match the expected native function's name. This second step is necessary because some detection scripts do not call *toString()* at all, they simply read *fn.name*, which simply returns the name of the function as a string, and check whether it matches what they expect for a native function.

Throughout the rest of the extension, every replaced function is passed through *hideFn* immediately after it is defined. This includes modifications on Navigator properties, Date methods, Canvas operations, WebGL parameters, Audio buffers, and the Worker constructor. As a result, no hooked function in the extension is distinguishable from its original version through either *toString()* inspection or name property checks.

B. Navigator

Chrome sets up many Web API properties as lazy accessors that only appear on the prototype when they are first accessed. Lazy accessors mean the API properties are not created if they

are never accessed. Defining them with Object.defineProperty before the first request would add them as new own properties rather than modify existing ones, changing the property enumeration order returned by Object.getOwnPropertyNames and showing an immediate sign of a modified structure.

The extension instead wraps *window.navigator* in a Proxy over the real navigator object. The proxy intercepts property reads through its *get trap*, a function that runs whenever someone tries to read a property, and returns spoofed values for the targeted properties (*userAgent, appVersion, platform, deviceMemory, hardwareConcurrency, language, languages, vendor, webdriver, userAgentData*). All other reads, along with the *ownKeys*, *getOwnPropertyDescriptor*, and *has traps*, are forwarded to the real navigator object. This way, *Navigator.prototype* stays completely untouched, and the native property order and descriptor identities are preserved.

This design has one known vulnerability. If a script accesses a navigator property by extracting its getter function directly through *Object.getOwnPropertyDescriptor* and calling it manually, it bypasses the Proxy and reaches the real, unmodified value. This happens because the extension intentionally forwards descriptor lookups to the real navigator object to avoid detection. In practice, this is a difficult attack to trigger, it requires working around the normal way of reading properties rather than simply accessing *navigator.userAgent*. For this reason, it is treated as an accepted trade-off, since it cannot be fully solved in pure JavaScript without support from the browser itself.

The *userAgentData* property returns a constructed object with a spoofed *getHighEntropyValues()* method. This method returns promises that resolve to the correct profile values, including architecture (x86), bitness (64), and a shuffled brands array matching the selected Chrome version. Both *getHighEntropyValues* and *toJSON* are registered as being native through *hideFn*.

C. Canvas

Image-based fingerprinting relies on the fact that subtle differences in how graphics hardware and drivers render the same drawing instructions produce slightly different pixel values, which can be hashed into an almost unique identifier. The extension intercepts the function used to read back pixel data from a drawing surface and perturbs a small number of pixel values by an imperceptible amount before returning them. Because this perturbation is drawn from the same deterministic sequence described in the Design section (III), the same drawing operation performed twice within a session remains exactly the same perturbed result, while the same operation performed in a different session returns a different one. A lookup structure tracks which surfaces have already had this perturbation applied, ensuring that a second read of the same surface returns the already modified values rather than applying a fresh layer of noise on top, an unnecessary and time-consuming action.

A related but distinct vector is the identification of the underlying graphics hardware itself, which scripts can query directly through a diagnostic extension to the graphics API. The function responsible for answering this query is replaced with one that, for the small number of parameters identifying the hardware vendor and model, returns a value selected from the session's chosen graphics-hardware identity rather than the device's real identity, while leaving every other parameter untouched so that the surrounding rendering behaviour continues to function normally.

D. Audio

A similar technique exists for audio: rendering a short sound through an offline audio-processing pipeline and reading back the resulting numeric waveform produces values that vary slightly across devices due to differences in audio hardware and processing precision, making this output usable as a fingerprint in much the same way as canvas output. The extension intercepts the function that exposes this rendered waveform and adds noise to a limited number of its values. Rather than modifying every sample in the buffer, the extension selects a random window covering 70%-90% of the buffer length. This protects against both a fingerprinting script that simply hashes the entire output and one that performs statistical analysis across the full buffer expecting an untouched signal, while keeping the perturbation small enough to be inaudible.

Fig. 1 and *Fig. 2* show the original and modified audio waveforms, respectively. The two are visually identical, confirming that the injected noise does not affect the perceived audio output. *Fig. 3* shows the difference between the two waveforms. The noise amplitude stays within the range of $\pm 2.0 * 10^{-5}$. This level of perturbation is far below the human hearing threshold, yet it is sufficient to change the audio fingerprint hash, since fingerprinting algorithms rely on exact sample-level comparisons. The audio noise is generated using a dedicated PRNG instance seeded from the session key.

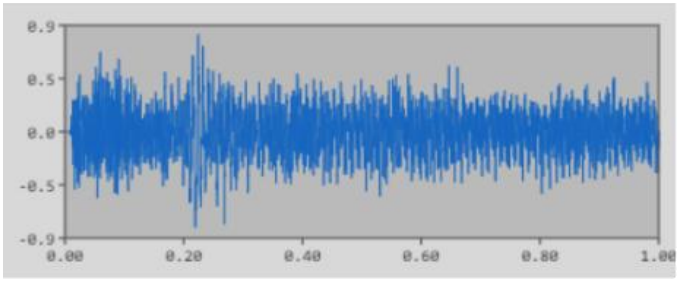

Fig. 1. Original waveform

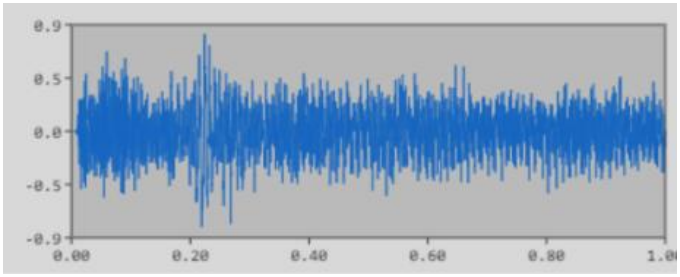

Fig. 2. Modified waveform

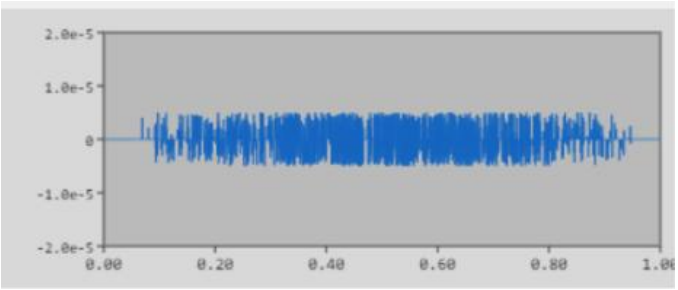

Fig. 3. Difference between the two waveforms

E. Timezone

The extension intercepts all *Date* and *Intl* APIs that expose timezone information. These APIs are connected to each other, so modifying only one creates mismatches that a fingerprinting script can detect by cross-referencing values from different sources.

The date formatting constructor is wrapped so that whenever a new date is created without an explicit timezone option, the extension silently injects the spoofed Internet Assigned Numbers Authority (IANA) zone name. The method that reports the resolved formatting options is also replaced to always return the spoofed zone. The substitute constructor preserves instance identity, so the returned object still passes type checks.

The method that returns the timezone offset returns the value from the selected profile. All getter methods for hours, minutes, seconds, date, day, month, and year apply a UTC-to-local shift using the fixed offset before returning a value.

The string representation methods for dates are implemented manually using modified getters, ensuring that the hour value matches the offset. This avoids using the date formatting API with a timezone name, which would apply real daylight saving time rules and produce a one-hour difference during winter months for zones that observe seasonal time shifts. This decision was made after a real test failure: the main thread correctly applied DST and gave UTC+1 for Berlin in January, while a background worker used the fixed offset and gave UTC+2, producing a visible mismatch between the two contexts.

F. WebGL

A diagnostic extension to the graphics API exposes two parameters that reveal the GPU manufacturer and model. The extension wraps the function responsible for answering these queries on both the WebGL1 and WebGL2 rendering contexts. When the queried parameter matches either of the two hardware identity constants, the replacement returns a vendor or renderer string from the selected GPU profile. All other queries are passed through unchanged.

The same GPU profile is used for both rendering context versions, so a site comparing the two cannot detect a mismatch.

G. WebRTC

WebRTC connection setup can leak the device's local and public IP address. During negotiation, the browser probes external servers to discover these addresses, and the results are available to page scripts without requiring any user permission.

The extension overrides the connection constructor. Regardless of what configuration the page provides, the extension forces the transport policy to be relay-only mode before the connection is created. In this mode, the browser does not attempt to discover its own network address. Instead, all traffic goes through an external relay server, so the user's real IP address is never exposed to the page. The constructor preserves instance identity, so the returned object still passes type checks.

H. Worker Injection

Web Workers run in a separate global context and do not inherit the modifications applied to the main page. A fingerprinting script that creates a Worker and queries browser properties from inside can bypass all the modifications made.

The extension handles this by overriding the Worker constructor. When a page tries to create a Worker, the override does not load the original script directly. Instead, it builds a script that contains two parts: first, a complete copy of all the spoofing modifications adapted for the Worker environment, and second, a call that loads the original worker script. The spoofed values *(user agent, timezone, GPU identity, languages, etc.)* are embedded directly into this script at construction time. This means the Worker sees the same profile as the main thread without needing any message passing between the two.

Inside the Worker, the spoofing modifications are applied to the Worker-specific navigator prototype using property overrides. The timezone, WebGL, and function masking modifications are also replicated.

Workers that use the newer module loading system are not covered by this approach, as the injection technique relies on the older script loading mechanism. This is a known limitation, though module-type Workers are rarely used by fingerprinting scripts in practice.

V. Evaluation

The extension was evaluated using three methods: a custom multi-user testing framework built with Playwright [10], the CreepJS fingerprinting tests, and the EFF's Cover Your Tracks [8] tool. Together, these methods assess three properties: whether the spoofed identity remains internally consistent within a session, whether it changes across sessions, and whether the user's fingerprint changes in each session.

A. Multi-User Test

We developed an automated testing framework using Playwright that simulates five independent users, each launching a separate Chromium browser instance with the

extension loaded. Each instance uses a fresh user data directory, ensuring a new session seed is generated every time. After loading a minimal test page served by a local HTTP server, the framework collects fingerprint values from every overridden API and runs a set of consistency and diversity checks.

The consistency checks verify that related properties agree with each other within a single session. For *Navigator* properties, the test confirms that the *User-Agent* string and the *User Agent Data* structure report the same Chrome version, that the platform is set to Windows, and that architecture and bitness are correctly reported. For *Screen* and *Window* properties, it verifies that the available dimensions do not exceed the full screen dimensions, that the outer window size matches the reported screen resolution, and that colour depth and pixel ratio are set to expected values. For *Timezone*, the test checks that the offset returned by *getTimezoneOffset* matches the offset embedded in the string returned by *Date.toString()* and *toTimeString()*, and that getter and setter methods for hours, months, and years return correct values. Canvas consistency is tested by reading pixel data twice from the same surface and confirming that both reads produce the same hash. *WebGL* consistency is verified by checking that WebGL1 and WebGL2 contexts report the same GPU identity. Audio consistency is confirmed by reading the same buffer twice and checking that both reads are identical. *WebRTC* is tested by checking that the transport policy is set to relay mode. Worker parity is verified by comparing every fake value read from inside a Worker with the corresponding value from the main thread, including user agent, platform, memory, CPU cores, language, timezone offset, timezone name, date string, and WebGL identity.

All consistency checks passed for all five users across every tested vector.

Group	Checks/user	Total
Navigator	6	30
Screen / Window	6	30
Timezone	8	40
Canvas	3	15
WebGL	5	25
Audio	2	10
WebRTC	1	5
Worker Parity	12	60
toString() Masking	13	65
Total	56	280

Fig. 4. Check groups and pass counts (5 simulated users).

The diversity checks measure how many distinct values appear across the five users for each property. The results show that canvas hash, FingerprintJS visitor ID and FingerprintJS audio hash each produced five distinct values out of five users, resulting in perfect diversity. Timezone zone names produced four distinct values. Screen resolution, memory, CPU cores, and language each produced three distinct values. Chrome version and WebGL GPU produced two distinct values each, which reflect the size of the current value pools (three Chrome profiles and four GPU profiles). The timezone UTC offset produced only one distinct value across all five users, because all timezone profiles in the current pool are Western or Central European zones that share the same offset during the summer months.

Property	Distinct values	Values observed
Chrome version	2	124, 122, 122, 124, 124
Language	3	It-IT, en-GB, pl-PL, it-IT, en-GB
Screen resolution	3	1366x768, 1440x900, 1440x900, 1440x900, 2560x1440
Memory	3	8, 8, 4, 8, 16 GB
CPU cores	3	8, 8, 4, 12, 12
Timezone zone	4	Warsaw, Rome, Warsaw, Berlin, Paris
Timezone offset	1	-120 for all 5
WebGL GPU	2	AMD RX 580 x4, NVIDIA GTX 1650 x1
Canvas hash	5	unique per session
Audio samples	5	unique per session

Fig. 5. Cross-session diversity across 5 simulated users.

The framework also integrates FingerprintJS, a commercial fingerprinting library, to verify that the extension produces a different visitor ID for each session. All five users received distinct visitor IDs, confirming that the extension successfully prevents cross-session tracking by a widely used production fingerprinting service.

B. CreepJS

CreepJS [6] is an open-source fingerprinting analyzer designed to detect spoofing and inconsistencies. It performs deep checks across multiple browser properties and flags values it considers suspicious or fake.

When tested with the extension active, CreepJS successfully blocked WebRTC, both host and Session Traversal Utilities for NAT (STUN) connections were reported as blocked, confirming that no local IP address was leaked. The timezone was flagged as spoofed, with CreepJS marking the reported offset and location as fake. This is expected: CreepJS compares the reported timezone against values it can infer from other signals, and since the extension replaces the real timezone with a different one, the mismatch is detected. This detection does not identify the user, it only indicates that a privacy tool is active.

The Screen test showed that CreepJS detected a mismatch between the spoofed screen dimensions and certain CSS media query values that the extension does not currently override. The JavaScript-level screen values were replaced correctly, but the browser's own CSS layout queries still reported the real screen dimensions, since these are controlled by the rendering engine and are not accessible to a content script.

The Window version test reported the fingerprint as "new," meaning CreepJS could not match it to any known browser fingerprint in its database. This is a positive result, the spoofed profile does not correspond to any real browser that CreepJS has seen, meaning it does not produce a recognisable or previously tracked identity.

C. Cover Your Tracks

Cover Your Tracks, maintained by the Electronic Frontier Foundation, measures how unique a browser's fingerprint is and whether it is protected against tracking. When tested with the extension active, the tool reported that the browser's fingerprint was randomised and that it had full protection against web tracking. This confirms that from the perspective of a standard fingerprinting analysis, the extension successfully prevents the browser from being uniquely identified.

D. Summary of Results

The evaluation confirms that the extension achieves its primary goals. First, all values remain internally consistent within a session, no mismatch was detected between related properties in any test. Second, the fingerprint changes across sessions, producing distinct profiles for each simulated user and preventing cross-session linking by both open-source and commercial fingerprinting tools.

The evaluation also reveals areas for improvement. The limited size of some value pools (three Chrome versions, four GPU profiles) means that with only five users, some properties do not achieve full diversity. The timezone offset pool produces no diversity at all during summer months. CSS-level screen queries are not currently covered, which creates a detectable mismatch in tools like CreepJS that cross-reference JavaScript values with CSS media queries.

VI. Conclusion

This paper presented a Chromium browser extension that defends against browser fingerprinting by replacing the user's real fingerprint with a coherent fake identity drawn from a pool of realistic profiles. The extension operates on two layers: HTTP header rewriting through the *declarativeNetRequest* API and *JavaScript* API interception through a content script injected before any page code executes. A seeded pseudo-random number generator ensures that all spoofed values remain consistent within a session, while a fresh seed is generated for each new session, preventing cross-session tracking.

The implementation covers eight fingerprinting vectors: Navigator properties, Screen and Window dimensions, Timezone, Canvas, WebGL, Audio, WebRTC, and Web Workers. Each vector required addressing specific technical challenges, from the lazy accessor behaviour of Chrome's Navigator prototype to the DST inconsistency between Intl-based and offset-based timezone representations. A function masking mechanism ensures that all replaced functions are indistinguishable from native browser functions through both *toString()* inspection and name property checks. Cross-frame propagation ensures that same-origin iframes see the same spoofed identity as the main page.

The evaluation demonstrated that the extension produces internally consistent fingerprints that change between sessions, successfully generating distinct visitor IDs in the FingerprintJS commercial library and achieving full protection according to the EFF's Cover Your Tracks tool.

Several limitations remain. The value pools are currently small, which limits the diversity of generated profiles. Font enumeration is not covered, leaving a potential fingerprinting vector open. CSS media queries report the real screen dimensions, creating a detectable mismatch with the spoofed JavaScript values. TLS-level fingerprinting operates at the network layer and cannot be addressed by a browser extension. Service Workers and module-type Web Workers are not covered by the current injection mechanism.

Future work could address these gaps by expanding the value pools with telemetry data from real-world browser populations, adding CSS-level overrides for screen dimensions, covering font enumeration, and exploring integration with browser-level APIs that could provide deeper access to properties currently outside the reach of a content script.

References

[1] P. Eckersley, "How unique is your web browser?" in Proc. 10th Int. Conf. Privacy Enhancing Technologies (PETS), 2010, pp. 1–18.

[2] S. Englehardt and A. Narayanan, "Online tracking: A 1-million-site measurement and analysis," in Proc. ACM Conf. Computer and Communications Security (CCS), 2016, pp. 1388–1401.

[3] Pierre Laperdrix, Benoit Baudry, Vikas Mishra. FPRandom: Randomizing core browser objects to break advanced device fingerprinting techniques. ESSoS 2017- 9th International Symposium on Engineering Secure Software and Systems, Jul 2017, Bonn, Germany. pp.17.

[4] The Tor Project, "Tor Browser Design Document," 2023. [Online]. Available: https://2019.www.torproject.org/projects/torbrowser/design/

[5] M. Gomez-Boix, P. Laperdrix, and B. Baudry, "Hiding in the crowd: An analysis of the effectiveness of browser fingerprinting at large scale," in Proc. WWW, 2018, pp. 309–318.

[6] A. Juliot, "CreepJS - Creep with JavaScript," GitHub, 2021. [Online]. Available: https://abrahamjuliot.github.io/creepjs/

[7] Brave Software, "Brave Browser Privacy Features," 2023. [Online]. Available: https://brave.com/privacy-features/

[8] Electronic Frontier Foundation, "Cover Your Tracks," 2020. [Online]. Available: https://coveryourtracks.eff.org/

[9] FingerprintJS Inc., "FingerprintJS - Browser Fingerprinting Library," GitHub, 2024. [Online]. Available: https://github.com/fingerprintjs/fingerprintjs

[10] Microsoft, "Playwright — End-to-End Testing Framework," 2024. [Online]. Available: https://playwright.dev/

[11] StatCounter, "Global Stats — Browser & Screen Resolution Market Share," Available: https://gs.statcounter.com

[12] Valve Corporation, "Steam Hardware & Software Survey,". Available: https://store.steampowered.com/hwsurvey

[13] AmIUnique, "Learn how identifiable you are on the Internet," Available: https://amiunique.org